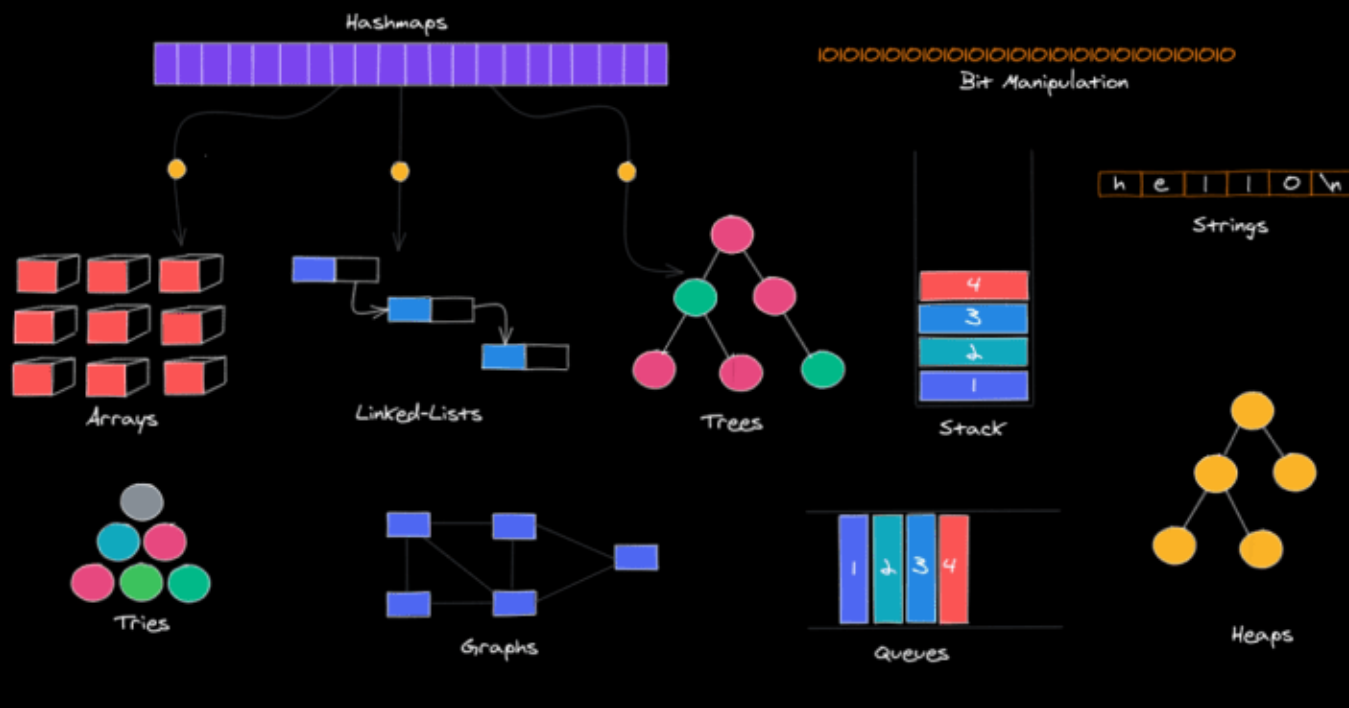




数据结构与算法导论

刘子博

FDSM

liuzibo@fudan.edu.cn

Today's Agenda

- 优先级队列
- 堆

Today's Agenda

- 优先级队列
- 堆

优先级队列



优先级队列

- 我们之前介绍过**队列**这一数据结构，其遵循**FIFO**（**先入先出**）的规则进行元素的插入和删除
- 然而，现实生活中有时我们需要一个除FIFO功能外还有额外删除功能的队列
 - 航空公司的候补等待（standby）队列中，优先级更高的乘客即使到的更晚，也有可能更早获得候补机会
- 我们引入**优先级队列**（**Priority Queue**）这一概念来描述这一类队列，其**删除**操作为删除具有**最高优先级**的元素

优先级队列的ADT

- 优先级队列中存储的为一个元素和其优先级的键-值对(key, value)
- 在优先级队列P上定义的优先级队列ADT支持以下方法：
 - `P.add(k, v)`: 向P中插入一个拥有键k和值v的元组
 - `P.remove_min()`: 从P中移除一个拥有最小键值的元组，并返回这个被移除的元组(k, v); 如果P为空则抛出错误
 - `P.min()`: 返回P中拥有最小键值的元组(k, v); 如P为空则抛出错误
 - `P.is_empty()`: 如果P不包含任何元组则返回True
 - `len(P)`: 返回P中的元组数量

例：

Operation	Return Value	Priority Queue
P.add(5,A)		{(5,A)}
P.add(9,C)		{(5,A), (9,C)}
P.add(3,B)		{(3,B), (5,A), (9,C)}
P.add(7,D)		{(3,B), (5,A), (7,D), (9,C)}
P.min()	(3,B)	{(3,B), (5,A), (7,D), (9,C)}
P.remove_min()	(3,B)	{(5,A), (7,D), (9,C)}
P.remove_min()	(5,A)	{(7,D), (9,C)}
len(P)	2	{(7,D), (9,C)}
P.remove_min()	(7,D)	{(9,C)}
P.remove_min()	(9,C)	{ }
P.is_empty()	True	{ }
P.remove_min()	"error"	{ }

全序关系

- 优先级队列中的键可以是定义了顺序的任意对象
- 优先级队列中的两个不同对象可以有相同的键
- 优先级队列中的键应有其对应的**全序关系**
- 一个集合 A 上的全序关系 $\leq$ 需满足：
 - **自反性**:
$$\forall x \in A, x \leq x$$
 - **反对称性**:
$$\forall x, y \in A, x \leq y \wedge y \leq x \Rightarrow x = y$$
 - **传递性**:
$$\forall x, y, z \in A, x \leq y \wedge y \leq z \Rightarrow x \leq z$$
 - A 中**任意**两个元素均可比较

优先级队列的实现

- 我们设计一个嵌套类_Item来存储键值对
- 我们使用一个双向链表来存储优先级队列，列表中的每个元素为一个_Item

优先级队列的实现

```
class PriorityQueueBase:
    """Abstract base class for a priority queue."""

    #----- nested _Item class -----
    class _Item:
        """Lightweight composite to store priority queue items."""
        __slots__ = '_key', '_value'

        def __init__(self, k, v):
            self._key = k
            self._value = v

        def __lt__(self, other):
            return self._key < other._key    # compare items based on their keys

    def is_empty(self):
        """Return True if the priority queue is empty."""
        return len(self) == 0
```

优先级队列的实现

- 使用未排序列表实现优先级队列：



- 各操作的时间复杂度：
 - 由于插入元素时只需要插到队尾，不需要进行顺序调整，故add操作需要的时间为 $O(1)$
 - 需要找到键最小的元素时，由于列表无序，故需要遍历整个列表，所以remove_min和min操作需要的时间为 $O(n)$

优先级队列的未排序列表实现

```
class UnsortedPriorityQueue(PriorityQueueBase): # base class defines _Item
    """A min-oriented priority queue implemented with an unsorted list."""

    #----- nonpublic behavior -----
    def _find_min(self):
        """Return Position of item with minimum key."""
        if self.is_empty(): # is_empty inherited from base class
            raise Empty('Priority queue is empty')
        small = self._data.first()
        walk = self._data.after(small)
        while walk is not None:
            if walk.element() < small.element():
                small = walk
            walk = self._data.after(walk)
        return small

    #----- public behaviors -----
    def __init__(self):
        """Create a new empty Priority Queue."""
        self._data = DoublyLinkedList()
```

优先级队列的未排序列表实现

```
def __len__(self):
    """Return the number of items in the priority queue."""
    return len(self._data)

def add(self, key, value):
    """Add a key-value pair."""
    self._data.add_last(self._Item(key, value))

def min(self):
    """Return but do not remove (k,v) tuple with minimum key.
    Raise Empty exception if empty.
    """
    p = self._find_min()
    item = p.element()
    return (item._key, item._value)

def remove_min(self):
    """Remove and return (k,v) tuple with minimum key.
    Raise Empty exception if empty.
    """
    p = self._find_min()
    item = self._data.delete(p)
    return (item._key, item._value)
```

优先级队列的实现

- 使用排序列表实现优先级队列：



- 各操作的时间复杂度：
 - 由于插入元素时需要扫描列表来找到合适的插入位置，故`add`操作需要的时间为 $O(n)$
 - 需要找到键最小的元素时，由于列表有序，故不需要遍历列表，所以`remove_min`和`min`操作只需要操作列表的第一个元素，所需要的时间为 $O(1)$

优先级队列的排序列表实现

```
class SortedPriorityQueue(PriorityQueueBase): # base class defines _Item
    """A min-oriented priority queue implemented with a sorted list."""

    #----- public behaviors -----

    def __init__(self):
        """Create a new empty Priority Queue."""
        self._data = DoublyLinkedList()

    def __len__(self):
        """Return the number of items in the priority queue."""
        return len(self._data)

    def add(self, key, value):
        """Add a key-value pair."""
        newest = self._Item(key, value) # make new item instance
        walk = self._data.last()       # walk backward looking for smaller key
        while walk is not None and newest < walk.element():
            walk = self._data.before(walk)
        if walk is None:
            self._data.add_first(newest) # new key is smallest
        else:
            self._data.add_after(walk, newest) # newest goes after walk
```

优先级队列的排序列表实现

```
def min(self):
    """Return but do not remove (k,v) tuple with minimum key.
    Raise Empty exception if empty.
    """
    if self.is_empty():
        raise Empty('Priority queue is empty.')
    p = self._data.first()
    item = p.element()
    return (item._key, item._value)

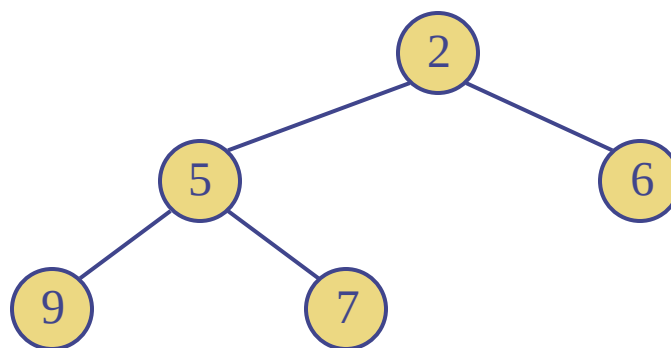
def remove_min(self):
    """Remove and return (k,v) tuple with minimum key.
    Raise Empty exception if empty.
    """
    if self.is_empty():
        raise Empty('Priority queue is empty.')
    item = self._data.delete(self._data.first())
    return (item._key, item._value)
```




Today's Agenda

- 优先级队列
- 堆

堆

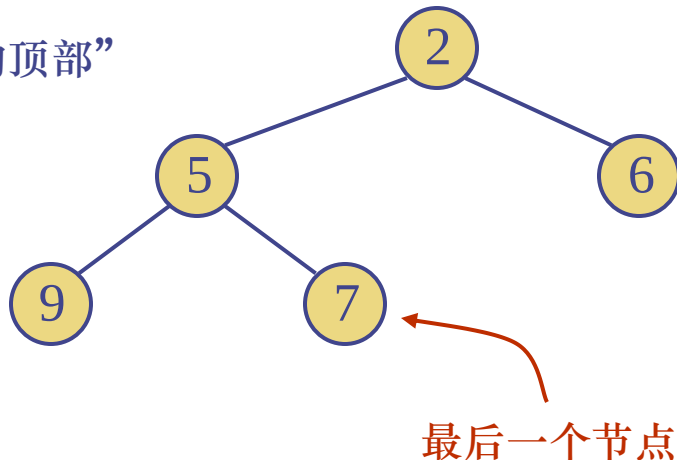


优先级队列的实现方式

- 我们使用位置列表实现优先级队列的两种方式展示了一个有趣的**权衡**:
 - 使用未排序列表存储键-值对时，我们能在 $O(1)$ 的时间内进行新元素的插入，但查找或删除有最小键的元组却需要 $O(n)$ 的时间
 - 使用排序列表存储键-值对时，我们可在 $O(1)$ 的时间内查找或删除有最小键的元组，但却需要 $O(n)$ 的时间进行新元素的插入
- 可否有更好的折中方式，即插入、查找或删除的时间相近，但均小于 $O(n)$?
 - 可以！我们使用一种被称为**堆**的数据结构来存储优先级队列

堆

- **堆 (heap)** 是一棵二叉树，其中的每个节点存储一个键-值对元组，且满足以下两条特殊性质：
 - **Heap-Order**属性：除根节点外，堆中的每个节点存储堆键值均大于或等于其父节点的键值，即 $key(v) \geq key(parent(v))$
 - ◆ 推论：键值最小的节点为根节点，“在堆的顶部”
 - **完全二叉树**属性：令 h 为堆的高度，则：
 - ◆ 对于 $i = 0, \dots, h - 1$, 第 i 层有 2^i 个节点
 - ◆ 在第 h 层，节点尽可能保存在最左侧的位置
 - ◆ 堆的最后一个节点在其最深一层的最右侧
 - ◆ 目的：为了提高效率，我们需要堆的高度尽可能小

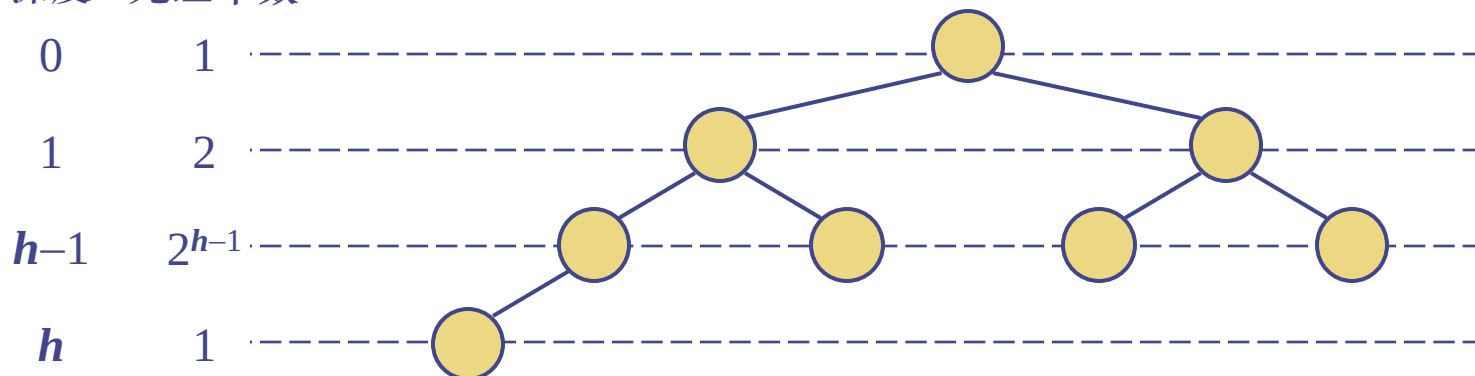


堆的高度



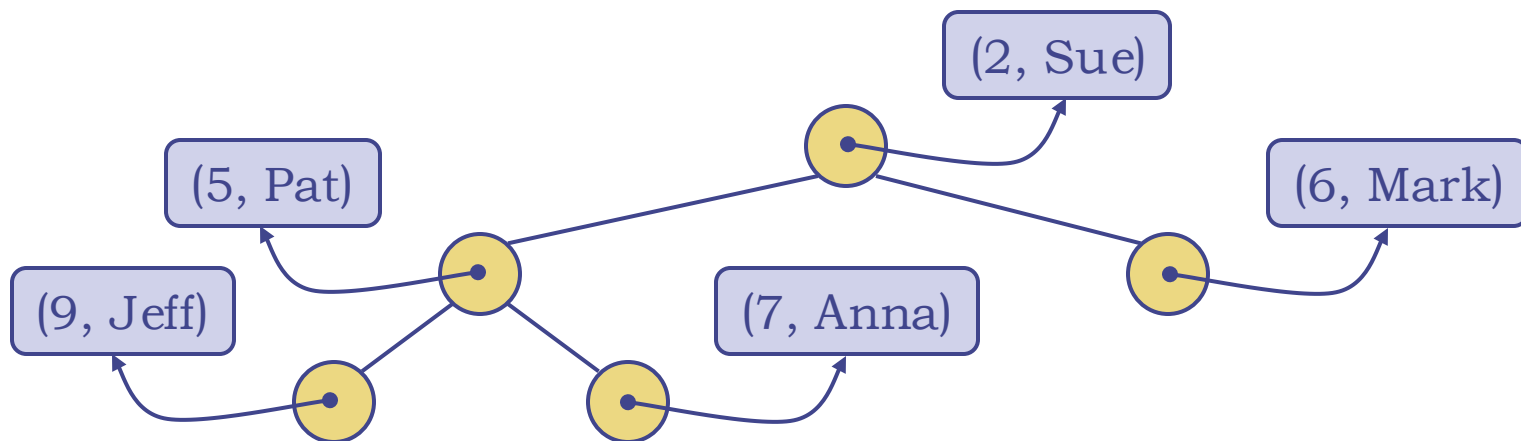
- 一棵存有 n 个元组的堆的高度为 $h = \lfloor \log n \rfloor$
 - 令 h 为存有 n 个元组的堆的高度
 - 由于堆为完全二叉树，故对于 $i = 0, \dots, h-1$ ，第 i 层有 2^i 个元组，所以有 $n \geq 1 + 2 + 4 + \dots + 2^{h-1} + 1 = 2^h$
 - 对于第 h 层，其节点最多有 2^h 个，故 $n \leq 1 + 2 + 4 + \dots + 2^{h-1} + 2^h = 2^{h+1} - 1$
 - 因此，我们有 $h = \lfloor \log n \rfloor$

深度 元组个数



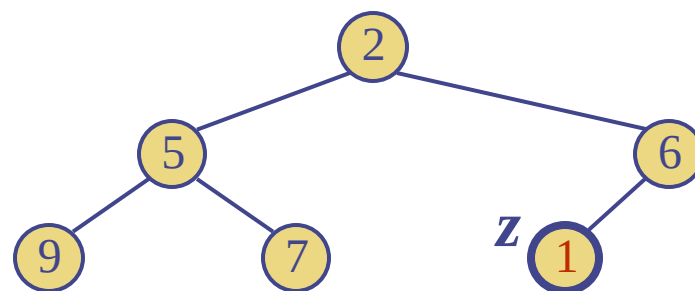
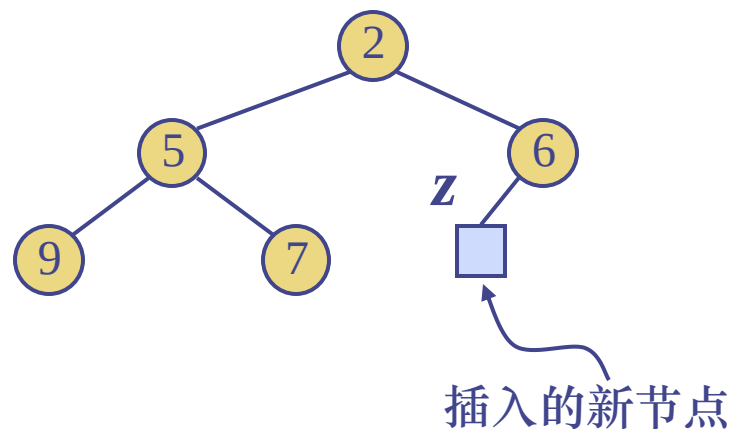
使用堆实现优先级队列

- 如果能以与堆的高度成比例的时间执行更新操作，则这些操作将在 $O(\log n)$ 时间内完成
- 如果使用堆实现优先级队列，如何实现优先级队列的插入新元素及查找和删除最高优先级元素的功能？
 - min**操作很简单，因为堆的属性保证了根节点的元组有最小键值



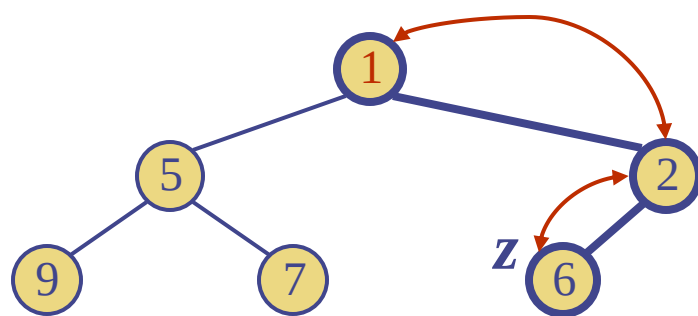
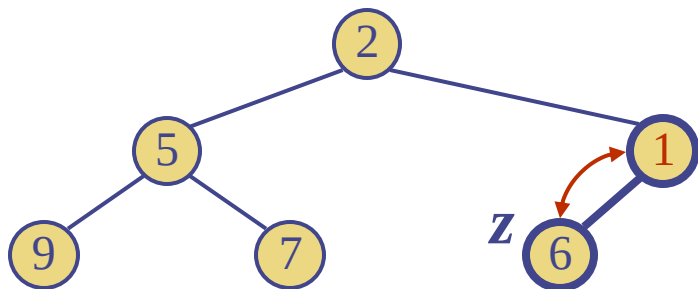
堆中元组的插入

- 优先级队列的 $\text{add}(k, v)$ 操作对应于在堆中**插入**一个新元组
- 堆中元素的插入由三个步骤组成：
 - 找到插入新节点的位置 z
 - 将新元组放在位置 z 处
 - 通过一系列操作恢复堆的 heap-order 属性

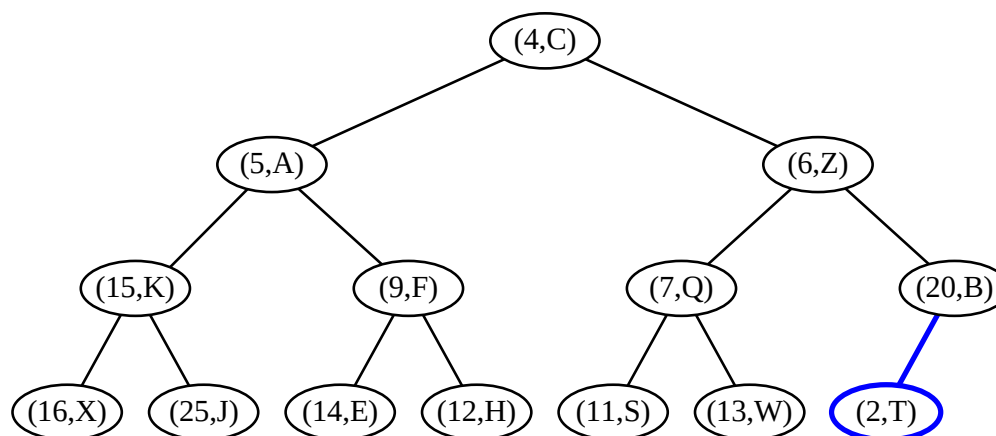
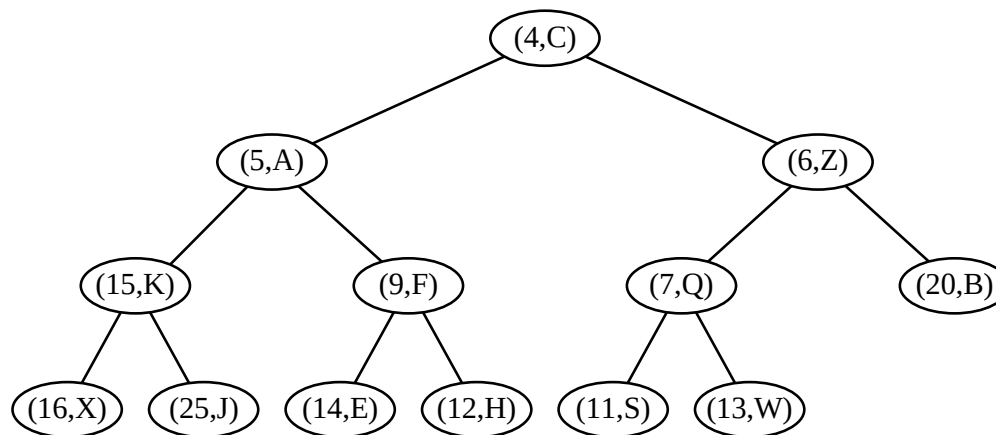


堆向上冒泡

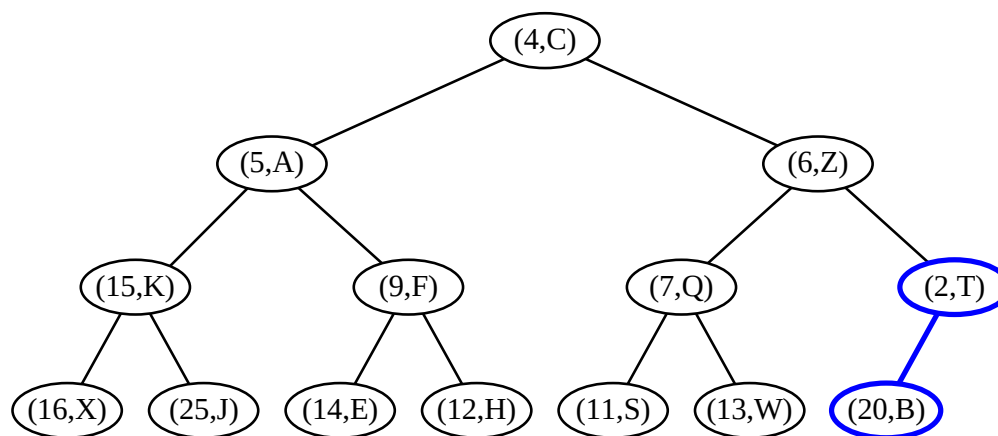
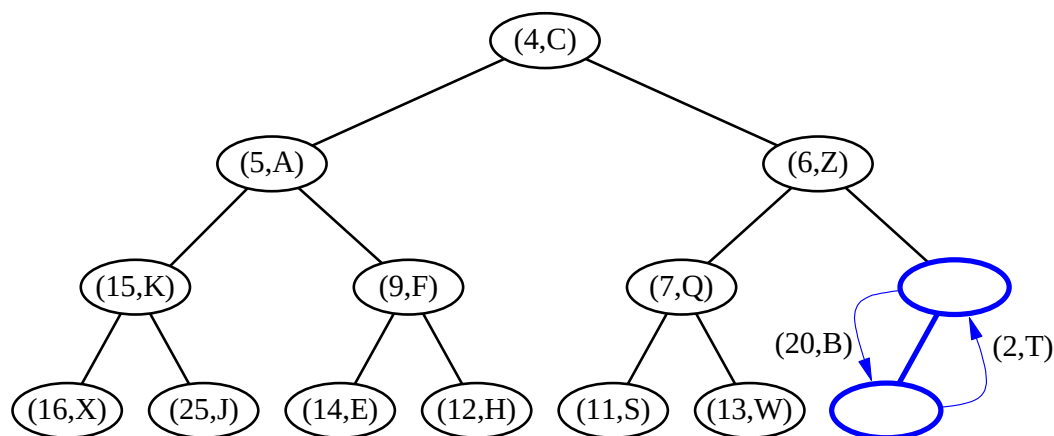
- 插入新元组后，堆可能变得不满足heap-order性质
- 通过对新节点和其父节点的键值进行比较，并在必要时交换两个节点的位置，可以使得最后两层节点的排布符合heap-order性质，但这可能使更高层的节点排布违背heap-order性质
- 通过不断将新节点沿着连接其与根节点的路径在必要时与其父节点交换，堆可以恢复其heap-order性质，这种操作被称为堆向上冒泡 (up-heap bubbling)
- 堆向上冒泡操作在新节点成为根节点或其键值大于其父节点键值时停止
- 堆向上冒泡操作的时间复杂度为 $O(\log n)$



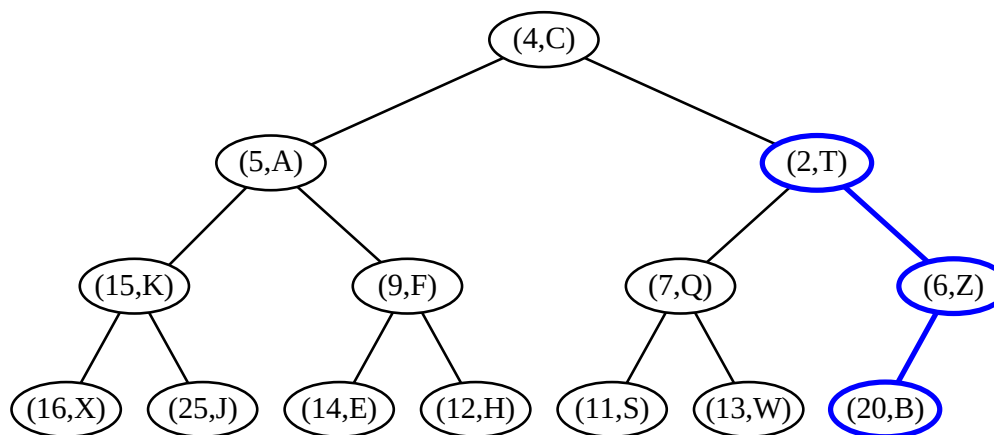
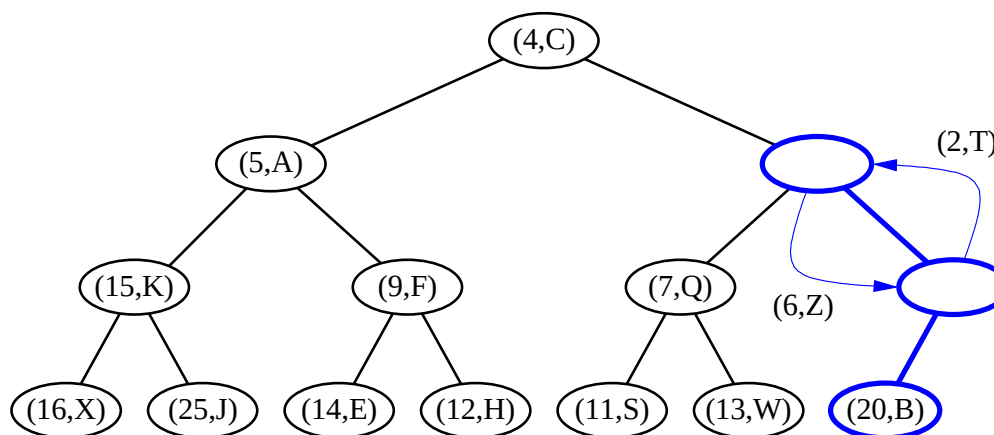
堆中元组的插入及向上冒泡



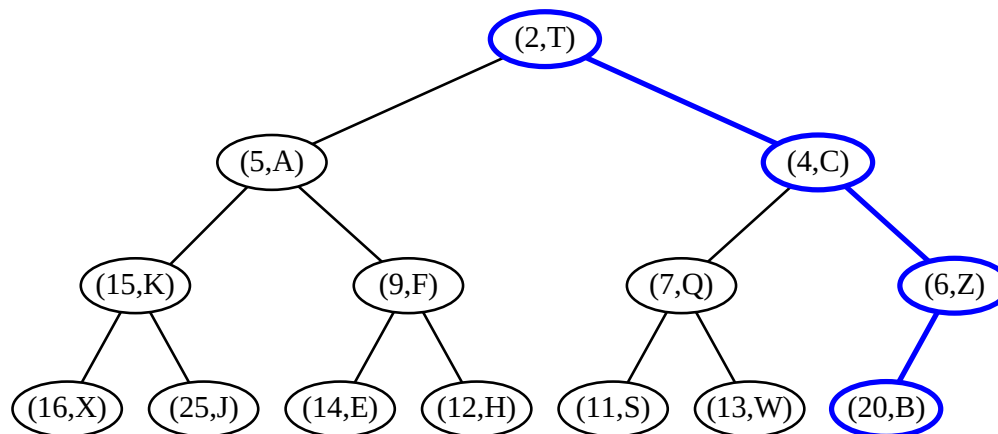
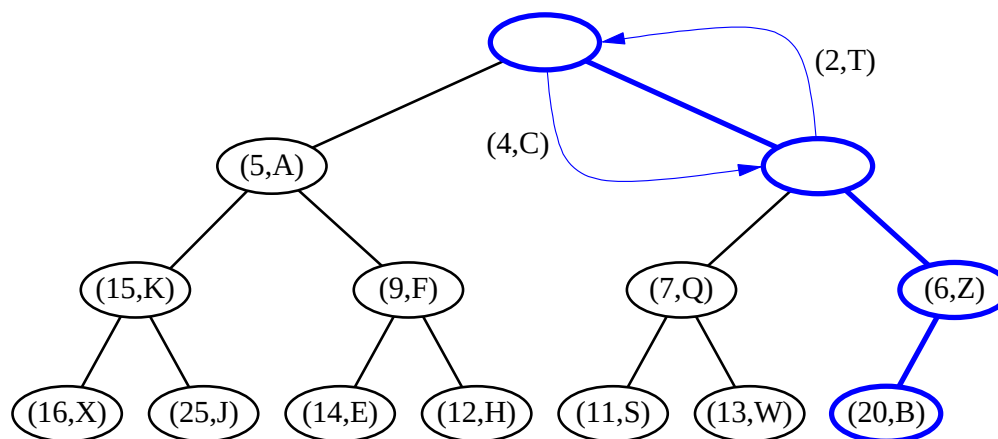
堆中元组的插入及向上冒泡



堆中元组的插入及向上冒泡

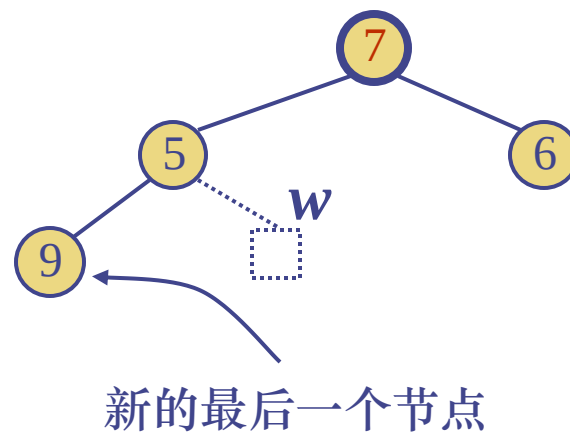
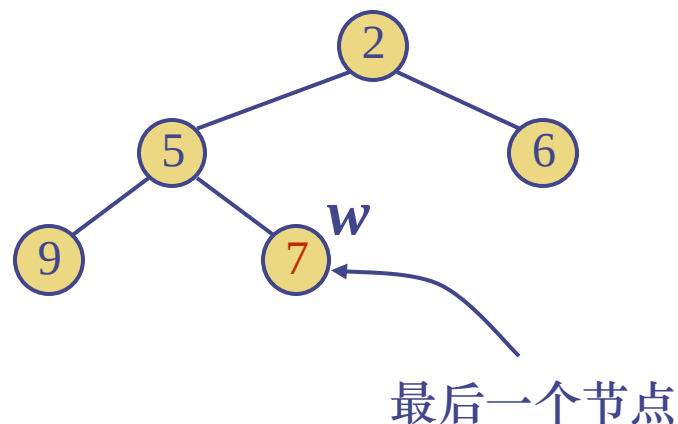


堆中元组的插入及向上冒泡



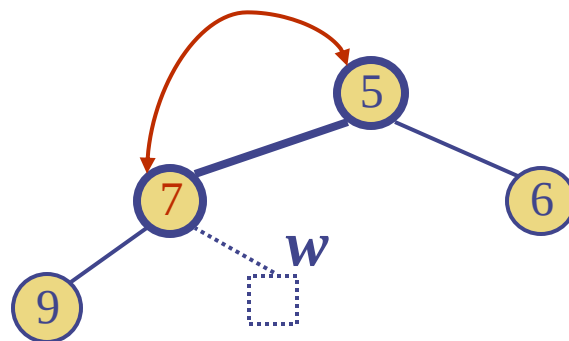
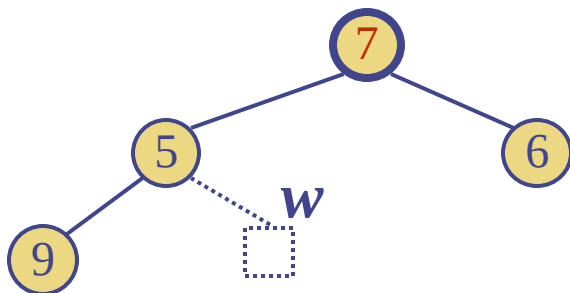
堆中元组的删除

- 优先级队列的`remove_min()`操作对应于在堆中删除根节点
- 堆中元素的删除由三个步骤组成：
 - 将根节点用最后一个节点 w 代替
 - 移除最后一个节点 w
 - 通过一系列操作恢复堆的 heap-order 属性

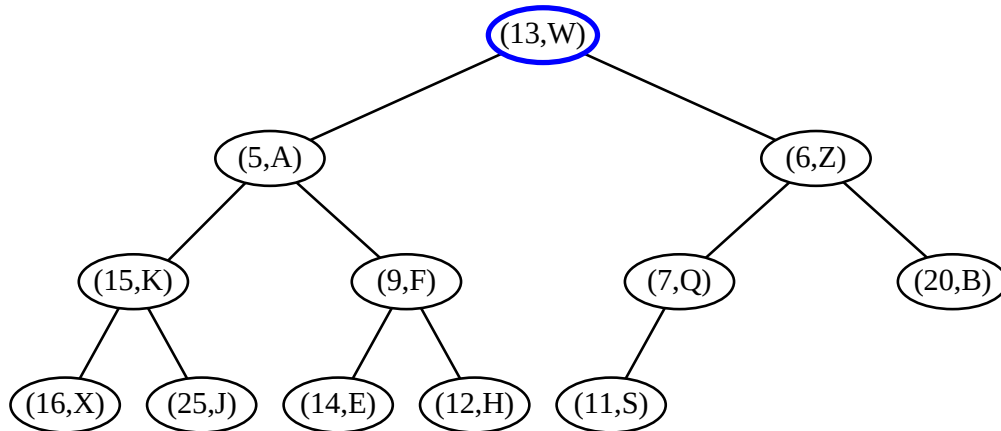
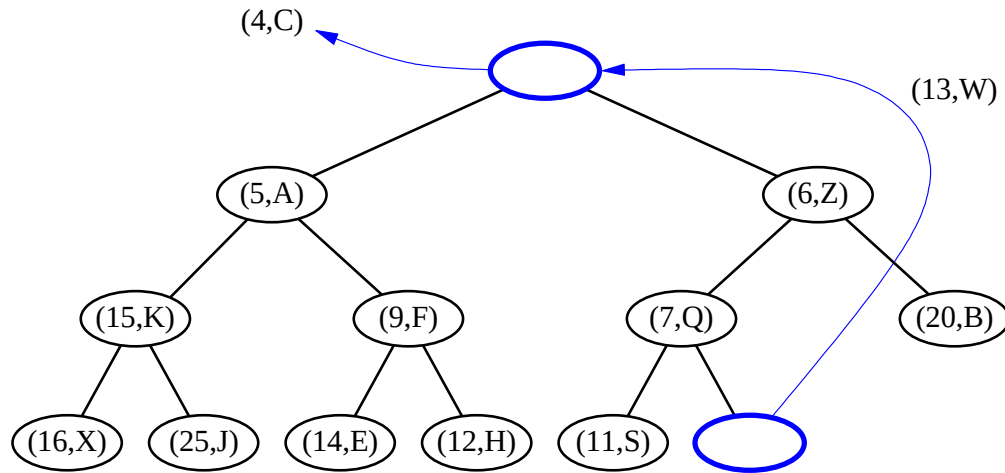


堆向下冒泡

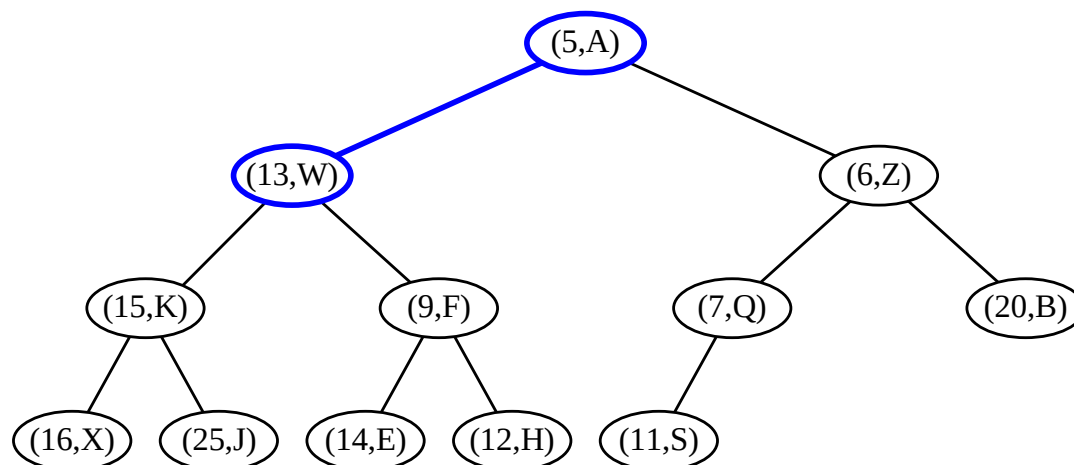
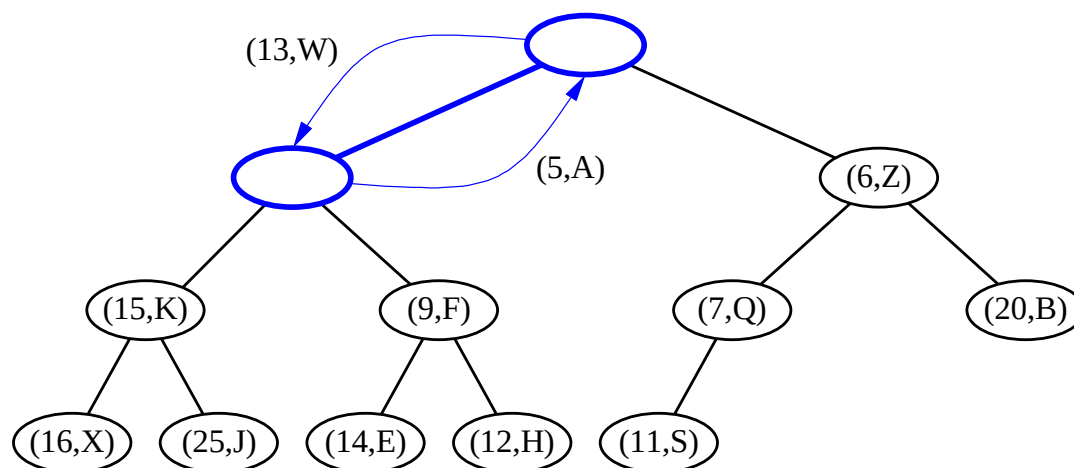
- 用最后一个节点将根节点替代后，堆可能变得不满足heap-order性质
- 通过对新的根节点和其孩子节点的键值进行比较，并在必要时**交换根节点和有较小键值的孩子节点的位置**，可以使得前两层节点的排布符合heap-order性质，但这可能使更低层的节点排布违背heap-order性质
- 通过不断将原来的最后一个节点在必要时向下与其孩子节点交换，堆可以恢复其heap-order性质，这种操作被称为**堆向下冒泡 (down-heap bubbling)**
- 堆向下冒泡操作在最后一个节点成为叶子节点或其键值小于其所有孩子节点键值时停止
- 堆向下冒泡操作的时间复杂度为 $O(\log n)$



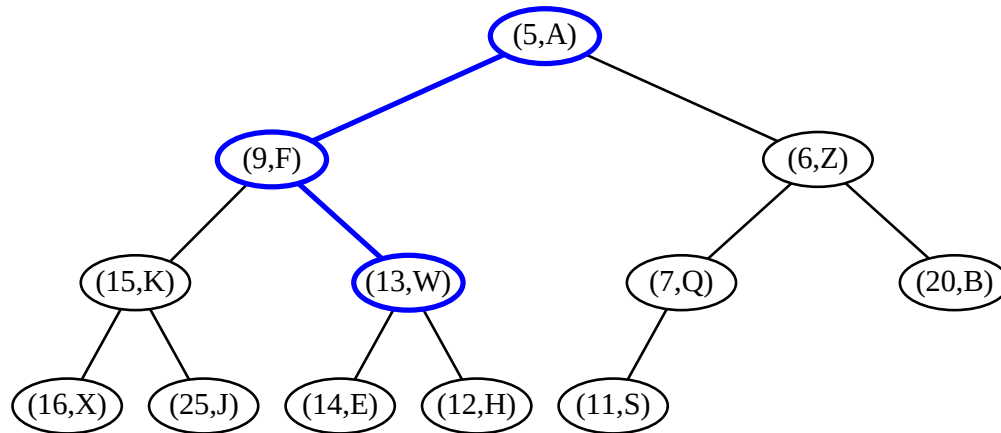
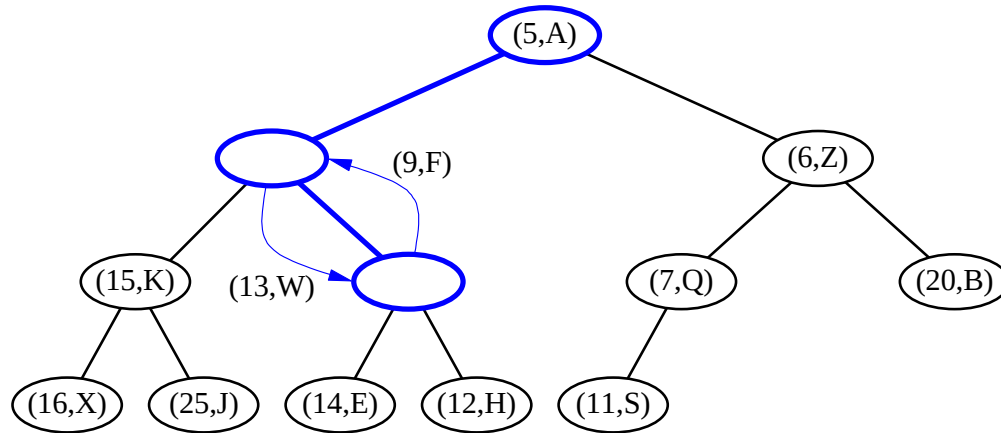
堆中元组的删除及向下冒泡



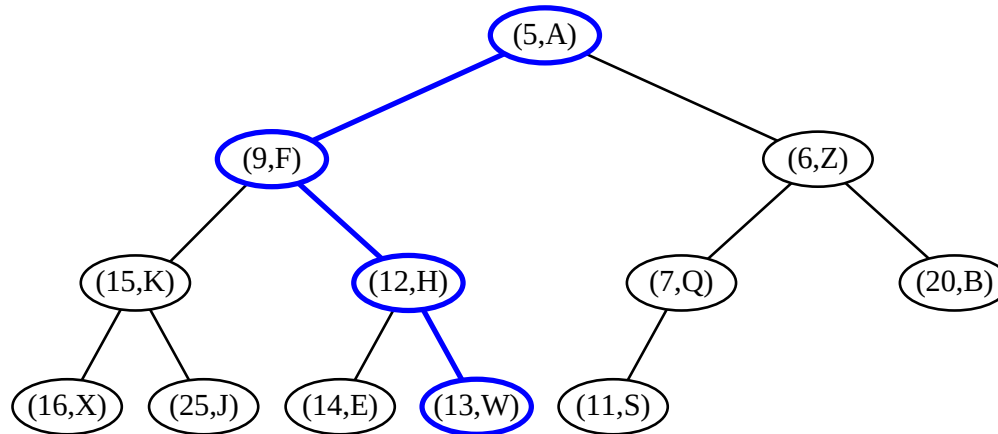
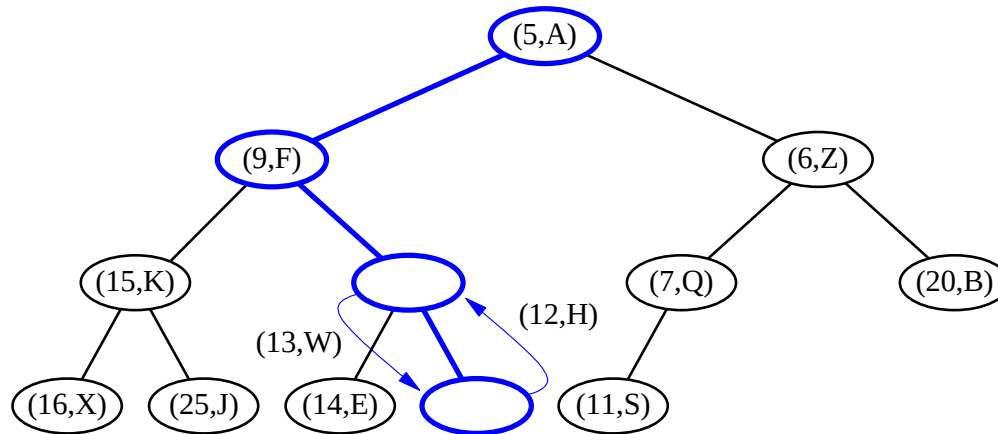
堆中元组的删除及向下冒泡



堆中元组的删除及向下冒泡

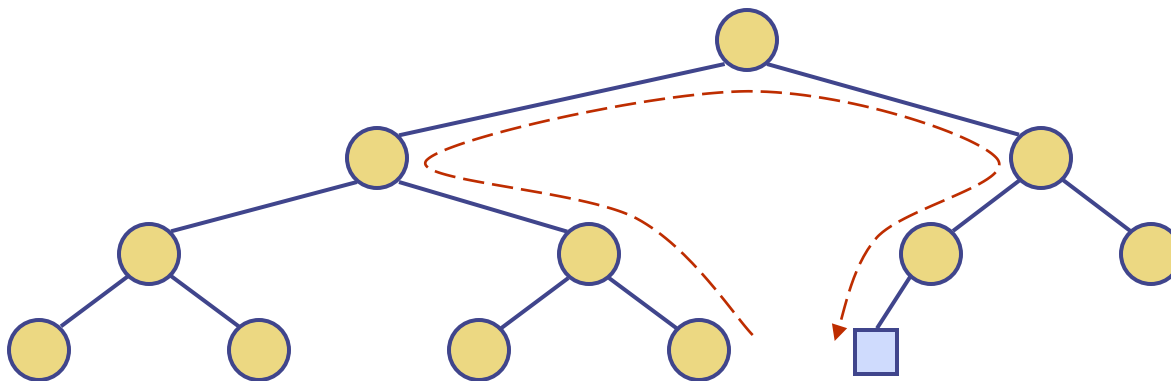


堆中元组的删除及向下冒泡



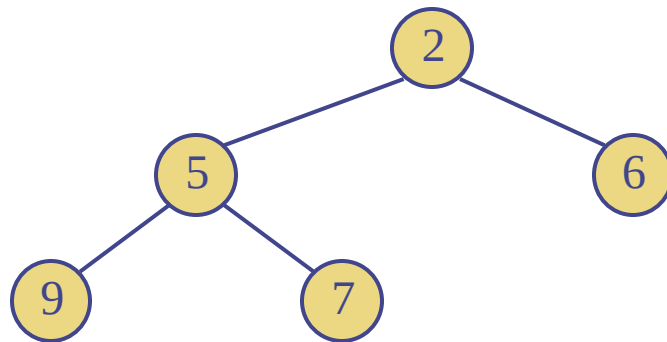
更新最后一个节点的位置

- 我们一般使用链式结构来存储树，则向堆中插入新节点时，新节点插入堆的位置可通过如下方法遍历 $O(\log n)$ 个节点得到：
 - 从最后一个节点的位置向上追溯，直到追溯到一个左孩子节点或根节点
 - 如果追溯到一个左孩子节点，则找到其兄弟（即右孩子节点）
 - 一直沿左侧路径向下，直到找到一个叶子节点
- 删除最高优先级节点后最后一个节点位置的更新与之类似
- 思考：是否有更好的实现堆的方式？



基于数组的完全二叉树表示

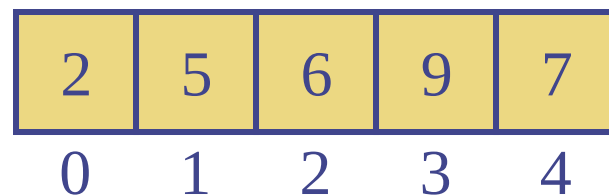
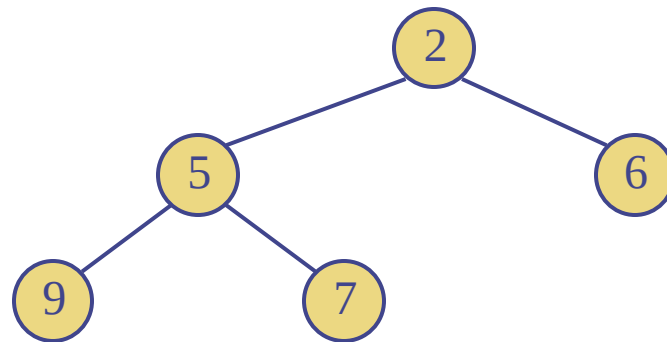
- 我们可以使用**数组**来表示完全二叉树，进而表示堆！
- 如在完全二叉树的数组表示中一节点的索引为 i ，则：
 - 其左孩子节点的索引为 $2i + 1$
 - 其右孩子节点的索引为 $2i + 2$
- 节点间的边并未被显式地存储
- 在数组表示的堆中，如堆中有 n 个元组，则这些元组恰好存在数组中的前 n 个位置上，且最后一个节点在索引 $n - 1$ 处



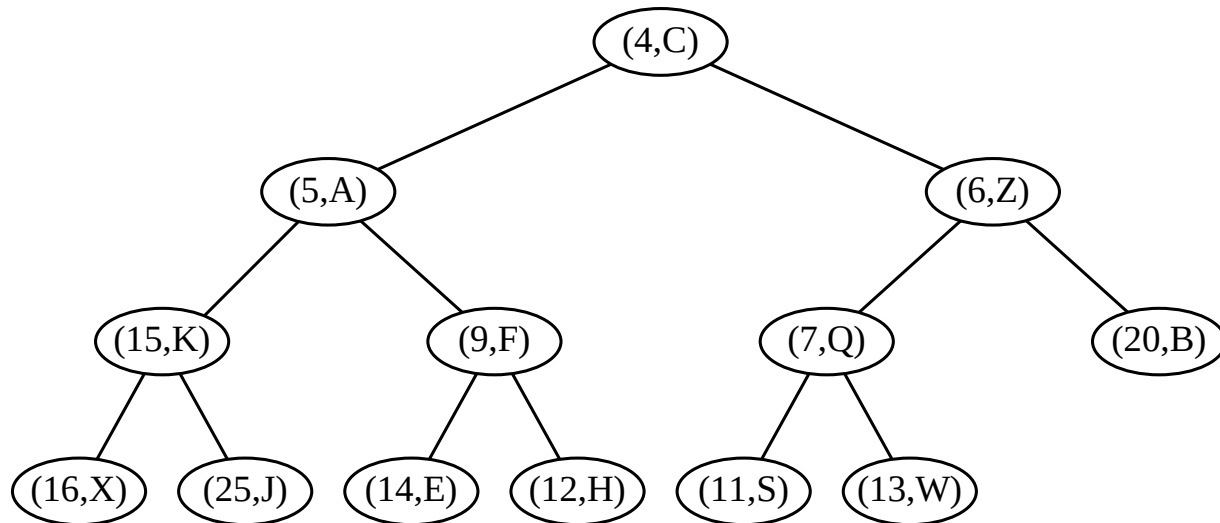
2	5	6	9	7
0	1	2	3	4

基于数组的完全二叉树表示

- 基于数组表示的堆使得我们避免了基于链式结构的堆结构的一些复杂性
- **add**操作对应于在数组中的索引 n 处加入一个新节点，并做堆向上冒泡操作
- **remove_min**操作对应于将索引 $n - 1$ 处的元素移动到根节点，并做堆向下冒泡操作



基于数组的完全二叉树表示



(4,C)	(5,A)	(6,Z)	(15,K)	(9,F)	(7,Q)	(20,B)	(16,X)	(25,J)	(14,E)	(12,H)	(11,S)	(8,W)
0	1	2	3	4	5	6	7	8	9	10	11	12

堆的性能分析

- 对于有 n 个节点的堆P:
 - 其空间利用率为 $O(n)$
 - 其各操作的时间复杂度为:

操作	运行时间
<code>P.min()</code>	
<code>P.add(k,v)</code>	
<code>P.remove_min()</code>	
<code>P.is_empty()</code>	
<code>len(P)</code>	

堆的性能分析

- 对于有 n 个节点的堆P:
 - 其空间利用率为 $O(n)$
 - 其各操作的时间复杂度为:

操作	运行时间
<code>P.min()</code>	$O(1)$
<code>P.add(k,v)</code>	$O(\log n)^*$
<code>P.remove_min()</code>	$O(\log n)^*$
<code>P.is_empty()</code>	$O(1)$
<code>len(P)</code>	$O(1)$

*如果是基于数组的，则为摊销结果

基于数组的堆实现

```
class HeapPriorityQueue(PriorityQueueBase): # base class defines _Item
    """A min-oriented priority queue implemented with a binary heap."""

    #----- nonpublic behaviors -----

    def _parent(self, j):
        return (j-1) // 2

    def _left(self, j):
        return 2*j + 1

    def _right(self, j):
        return 2*j + 2

    def _has_left(self, j):
        return self._left(j) < len(self._data)      # index beyond end of list?

    def _has_right(self, j):
        return self._right(j) < len(self._data)     # index beyond end of list?
```

基于数组的堆实现

```
def _swap(self, i, j):
    """Swap the elements at indices i and j of array."""
    self._data[i], self._data[j] = self._data[j], self._data[i]

def _upheap(self, j):
    parent = self._parent(j)
    if j > 0 and self._data[j] < self._data[parent]:
        self._swap(j, parent)
        self._upheap(parent)           # recur at position of parent

def _downheap(self, j):
    if self._has_left(j):
        left = self._left(j)
        small_child = left              # although right may be smaller
        if self._has_right(j):
            right = self._right(j)
            if self._data[right] < self._data[left]:
                small_child = right
        if self._data[small_child] < self._data[j]:
            self._swap(j, small_child)
            self._downheap(small_child) # recur at position of small child
```

基于数组的堆实现

```
#----- public behaviors -----  
def __init__(self):  
    """Create a new empty Priority Queue."""  
    self._data = []  
  
def __len__(self):  
    """Return the number of items in the priority queue."""  
    return len(self._data)  
  
def add(self, key, value):  
    """Add a key-value pair to the priority queue."""  
    self._data.append(self._Item(key, value))  
    self._upheap(len(self._data) - 1)           # upheap newly added position
```

基于数组的堆实现

```
def min(self):
    """Return but do not remove (k,v) tuple with minimum key.
    Raise Empty exception if empty.
    """
    if self.is_empty():
        raise Empty('Priority queue is empty.')
    item = self._data[0]
    return (item._key, item._value)

def remove_min(self):
    """Remove and return (k,v) tuple with minimum key.
    Raise Empty exception if empty.
    """
    if self.is_empty():
        raise Empty('Priority queue is empty.')
    self._swap(0, len(self._data) - 1)
    item = self._data.pop()
    self._downheap(0)
    return (item._key, item._value)
```

put minimum item at the end
and remove it from the list;
then fix new root



优先级队列排序

- 我们可以使用优先级队列对一个可比较元素集合进行排序
 - 首先使用`add`操作将集合中的元素一个接一个地插入到优先级队列中
 - 再使用`remove_min`操作将元素以某种顺序从优先级队列中移出
- 此算法的运行时间取决于优先级队列的具体实现方式

Algorithm *PQ-Sort*(*S*, *C*)

Input sequence *S*, comparator *C* for the elements of *S*

Output sequence *S* sorted in increasing order according to *C*

P $\leftarrow$ priority queue with comparator *C*

while $\neg S.is_empty()$

$e \leftarrow S.remove_first()$

$P.add(e, \emptyset)$

while $\neg P.is_empty()$

$e \leftarrow P.remove_min().key()$

$S.add_last(e)$

选择排序

- 当使用未排序列表实现优先级队列时，PQ-sort即为**选择排序**
- 选择排序的运行时间：
 - 将所有元素使用add操作插入优先级队列需要 n 次操作，运行时间为 $O(n)$
 - 将元素按顺序移出长为 n 的优先级队列需要每次使用remove_min操作**选择**优先级最高的元素，其运行时间正比于：

$$n + (n - 1) + \dots + 1$$

- 选择排序的运行时间复杂度为 $O(n^2)$

选择排序

Input:

Sequence S
(7,4,8,2,5,3,9)

Priority Queue P
()

Phase 1

(a) (4,8,2,5,3,9)

(7)

(b) (8,2,5,3,9)

(7,4)

..

.. ..

(g) () (7,4,8,2,5,3,9)

Phase 2

(a) (2) (7,4,8,5,3,9)

(b) (2,3) (7,4,8,5,9)

(c) (2,3,4) (7,8,5,9)

(d) (2,3,4,5) (7,8,9)

(e) (2,3,4,5,7) (8,9)

(f) (2,3,4,5,7,8) (9)

(g) (2,3,4,5,7,8,9) ()

插入排序

- 当使用排序列表实现优先级队列时，PQ-sort即为**插入排序**
- 插入排序的运行时间：

- 将所有元素使用add操作**插入**优先级队列的运行时间正比于：

$$1 + 2 + \dots + n$$

- 将排好序的元素使用remove_min操作从优先级队列中移出需要 n 次操作，运行时间为 $O(n)$
- 插入排序的运行时间复杂度为 **$O(n^2)$**

插入排序

Input:

Sequence S
(7,4,8,2,5,3,9)

Priority queue P
()

Phase 1

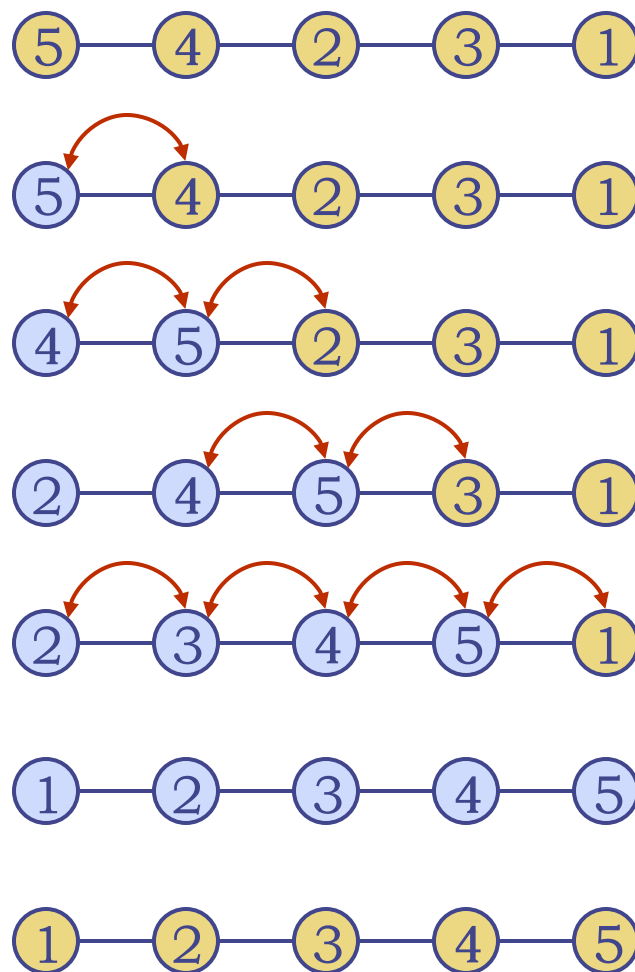
(a)	(4,8,2,5,3,9)	(7)
(b)	(8,2,5,3,9)	(4,7)
(c)	(2,5,3,9)	(4,7,8)
(d)	(5,3,9)	(2,4,7,8)
(e)	(3,9)	(2,4,5,7,8)
(f)	(9)	(2,3,4,5,7,8)
(g)	()	(2,3,4,5,7,8,9)

Phase 2

(a)	(2)	(3,4,5,7,8,9)
(b)	(2,3)	(4,5,7,8,9)
..	..	..
(g)	(2,3,4,5,7,8,9)	()

原地排序

- 对于选择排序和插入排序，我们可以实现**原地排序**，而不需要借助外部数据结构
- 输入序列的一部分即成为优先级队列
- 例：原地插入排序
 - 将序列的前半部分作为已排好序的部分
 - 使用元素**交换**对序列进行操作





优先级队列排序

- 我们可以使用优先级队列对一个可比较元素集合进行排序
 - 首先使用`add`操作将集合中的元素一个接一个地插入到优先级队列中
 - 再使用`remove_min`操作将元素以某种顺序从优先级队列中移出
- 此算法的运行时间取决于优先级队列的具体实现方式
- 选择/插入排序: $O(n^2)$
- 我们可以做到更好吗?

Algorithm *PQ-Sort*(S, C)

Input sequence S , comparator C for the elements of S

Output sequence S sorted in increasing order according to C

$P \leftarrow$ priority queue with comparator C

while $\neg S.is_empty()$

$e \leftarrow S.remove_first()$

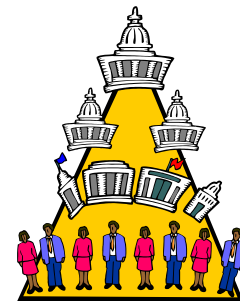
$P.add(e, \emptyset)$

while $\neg P.is_empty()$

$e \leftarrow P.remove_min().key()$

$S.add_last(e)$

堆排序



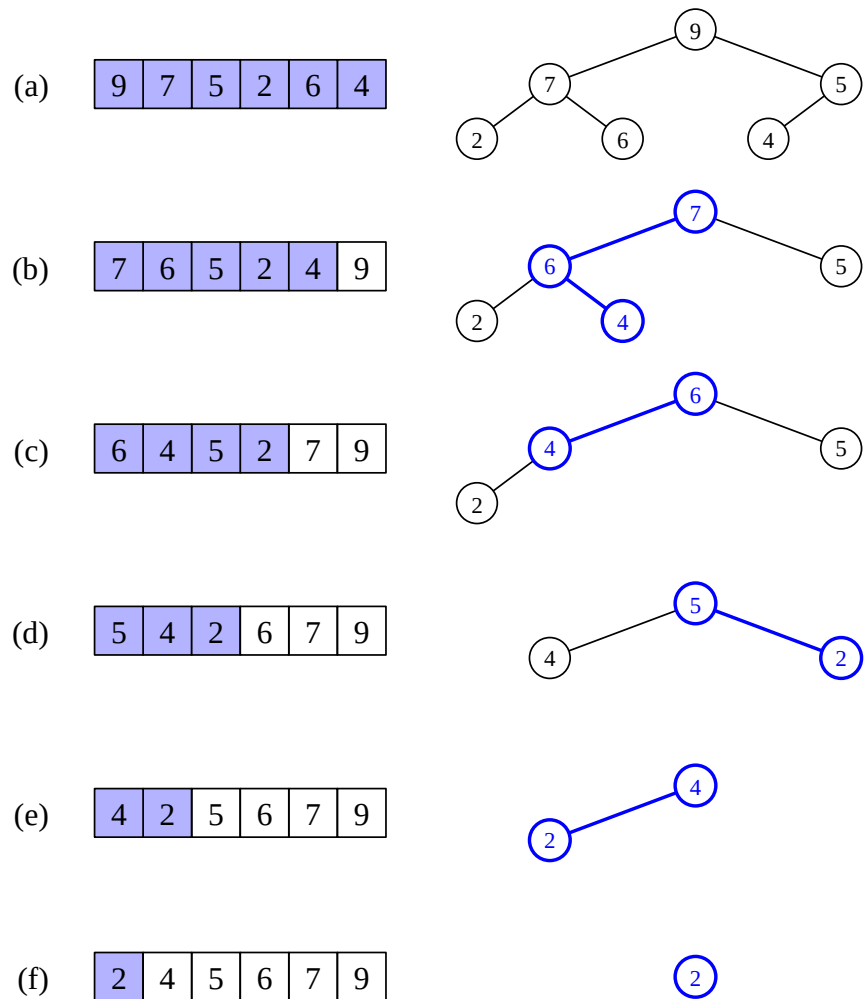
- 考虑一个有 n 个元素的使用堆实现的优先级队列：
 - 其空间复杂度为 $O(n)$
 - `add`和`remove_min`操作的运行时间均为 $O(\log n)$
 - `len`、`is_empty`和`min`操作的运行时间均为 $O(1)$
- 使用基于堆实现的优先级队列，我们可以对 n 个元素在 $O(n \log n)$ 时间内进行排序
- 这个排序算法被成为堆排序
- 堆排序比在二次时间内完成排序的算法（如选择排序和插入排序）快得多

原地堆排序

- 如堆是基于数组实现的，则我们可以将待排序列表的一部分作为堆来实现**原地堆排序**，具体步骤如下：
 - 重新定义堆的操作，使其成为**最大堆**（**maximum-oriented heap**），即父节点键值不小于孩子节点；算法执行过程中，始终使用列表的左半部分表示**堆**，右半部分表示**序列**
 - 在算法的第一阶段，我们从一个空堆开始，**从左向右**移动堆与序列之间的边界，一次一步；每一步将序列中的下一个元素追加到堆中
 - 在算法的第二阶段，我们从一个空的序列开始，**从右向左**移动堆与序列之间的边界，一次一步；每一步将最大值元素从堆中移除并存储到当前序列的最前方

原地堆排序

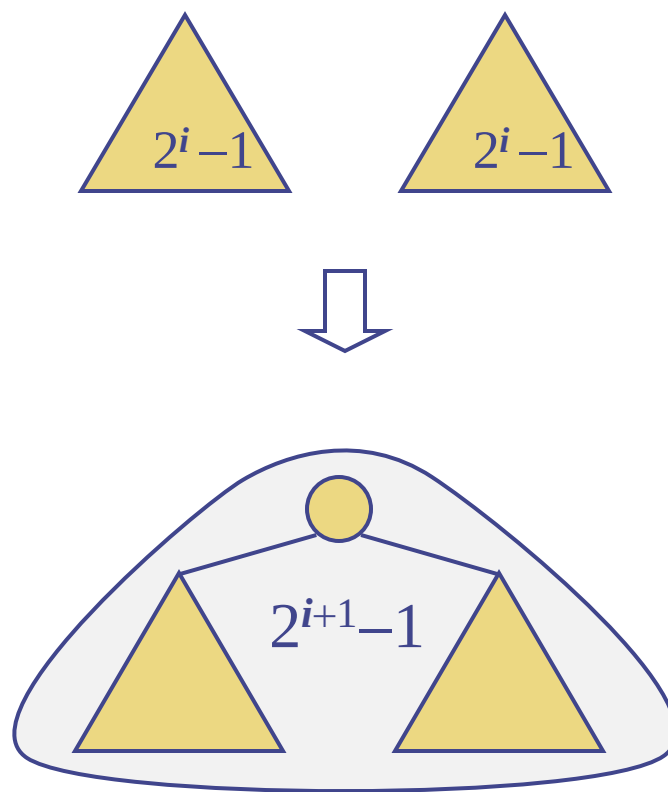
- 原地堆排序第二阶段示例：



自底向上构建堆

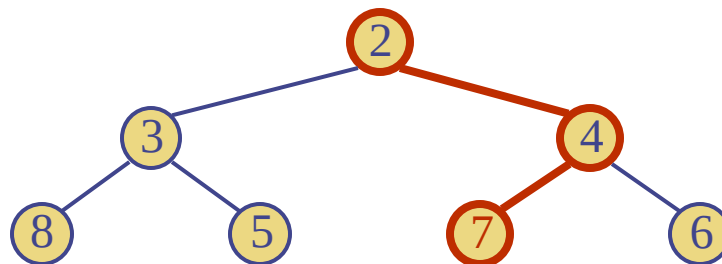
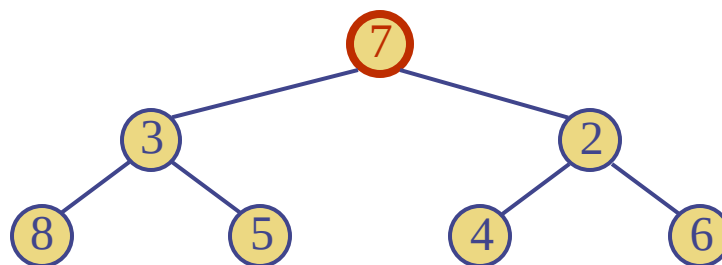


- 从一个空堆开始连续调用 n 次 `add` 操作的时间复杂度为 $O(n \log n)$ ；自底向上构建堆可将此操作的时间复杂度降为 $O(n)$
- 为了叙述简单，我们设 $n = 2^{h+1} - 1$ ，即堆为满二叉树。这时我们可以通过 $h + 1 = \log(n + 1)$ 个步骤自底向上构建堆
 - 第一步，构建 $(n + 1)/2$ 个基本堆，每个堆仅存储一个元素；
 - 第 i 步，将每两个有 $2^i - 1$ 个元素的堆合并为一个有 $2^{i+1} - 1$ 个元素的堆

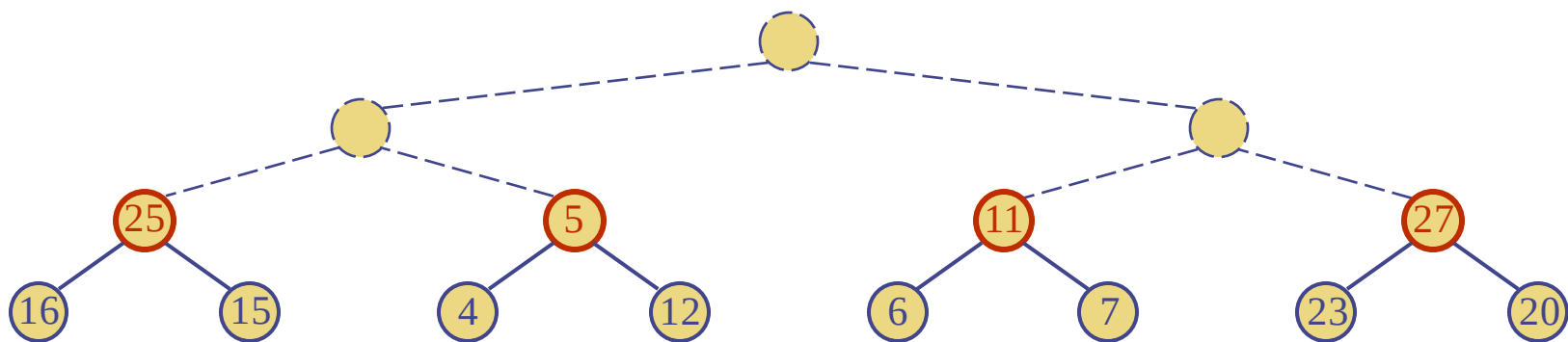
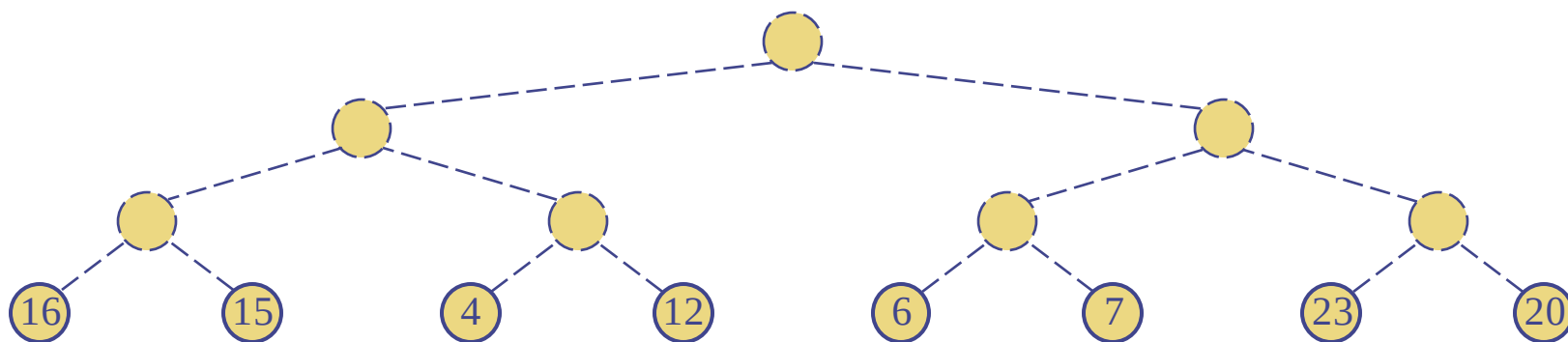


堆的合并

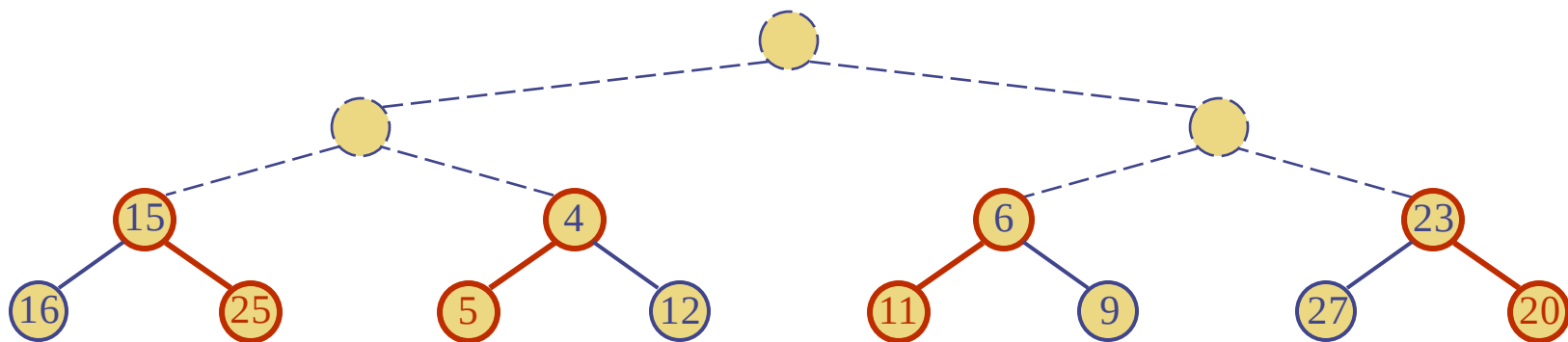
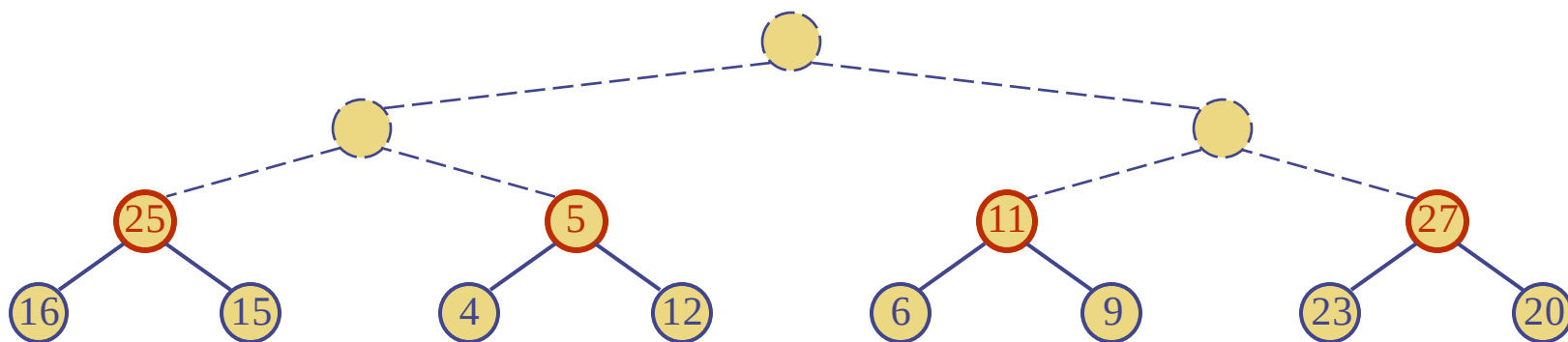
- 给定两个堆，以及一个新键值 k
- 创建一个以存储新键值 k 的节点为根的新堆，并将原来的两个堆作为新堆根节点的左右子树
- 进行堆向下冒泡操作恢复 heap-order 属性



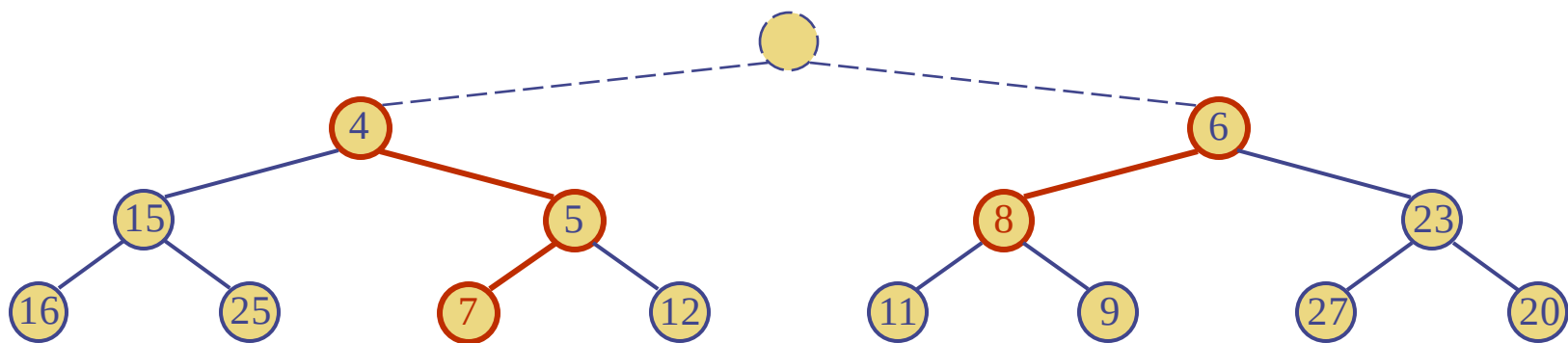
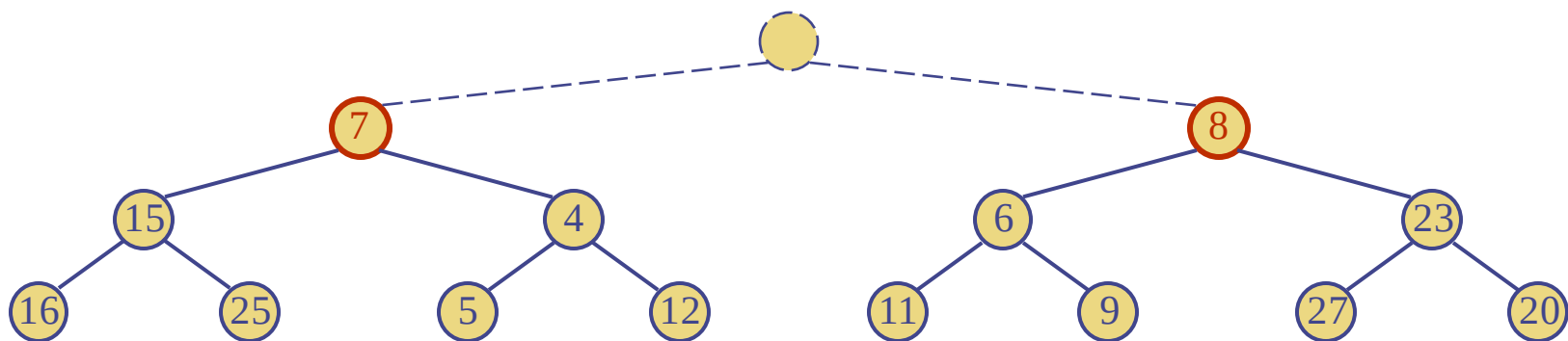
自底向上构建堆



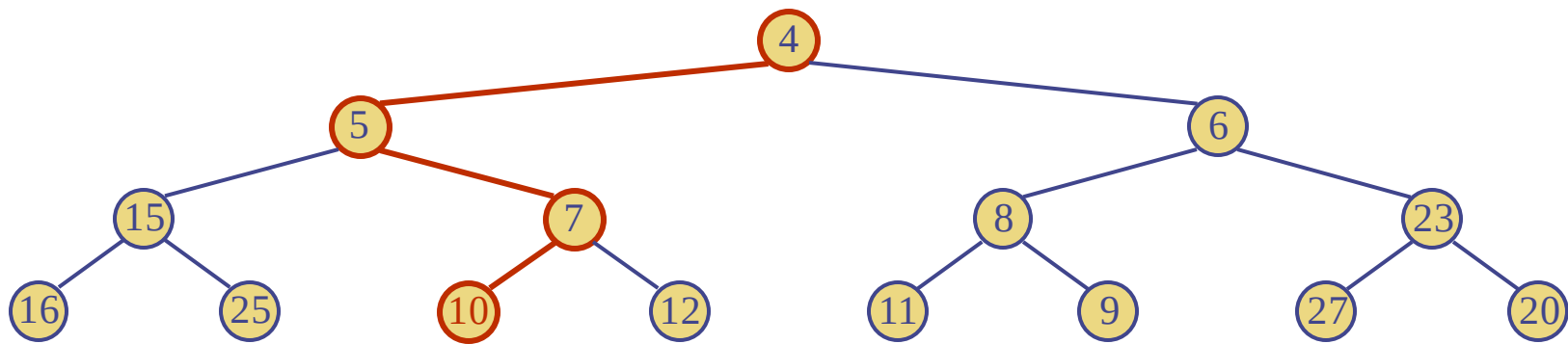
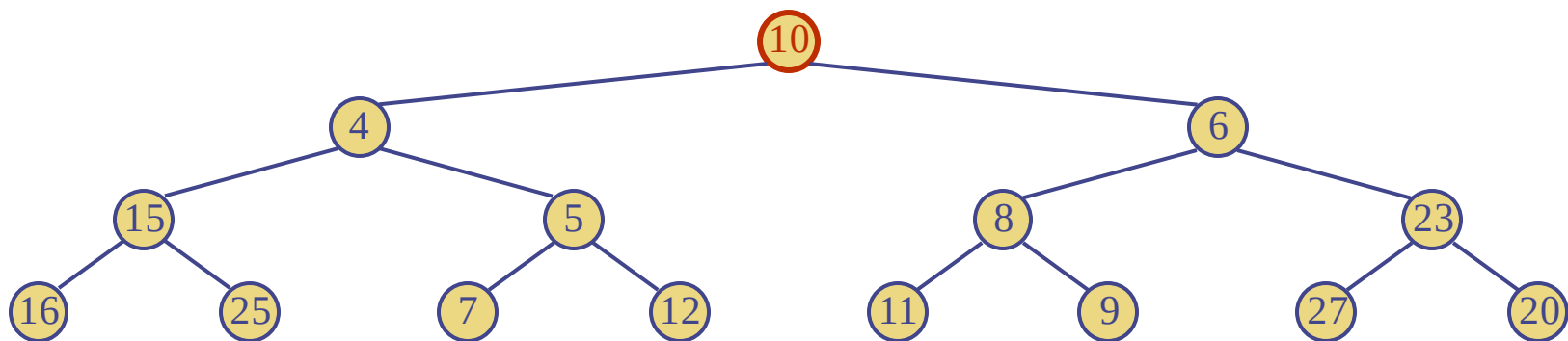
自底向上构建堆



自底向上构建堆

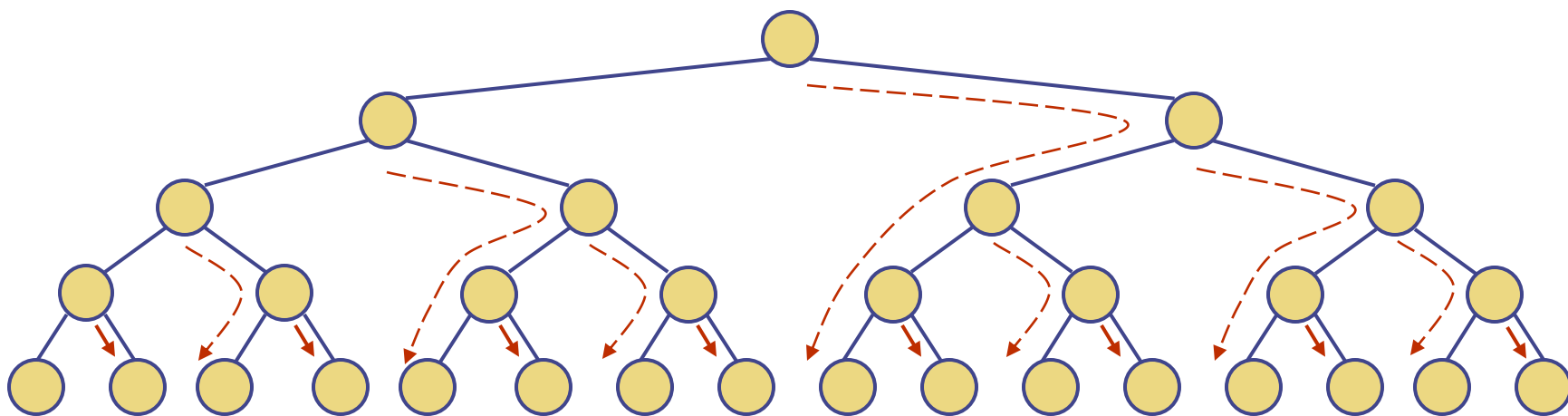


自底向上构建堆



自底向上构建堆的渐进分析

- 我们将最坏情况下的堆向下冒泡时间可视化，使得每个节点处的堆向下冒泡路径均先向右走，然后一直向左走，直到堆底部（此路径可能与实际的堆向下冒泡路径不同）
- 每个节点最多被两条路径经过，因此所有路径经过的边数总和为 $O(n)$
- 因此，自底向上构建堆的时间为 $O(n)$





Enjoy